

Análisis de MFA y OTP

Objetivos de aprendizaje

Modelar el estado pre-MFA y autenticado, comprobar binding de usuario/sesión/desafío, detectar replay y manipulación de estado, y descartar bypasses aparentes.

Prerrequisitos

Sesiones, autenticación, autorización, HTTP multipart/JSON y controles de repetición.

Fundamentos técnicos

El servidor debe verificar el factor y realizar la transición de estado. Un OTP debe tener vigencia limitada, límites de intentos y uso único. El estado no debe depender de una bandera enviada por el cliente. Además, desafío, usuario y sesión deben estar ligados para impedir que un código válido se aplique a otro flujo.

Modelo mental

```
credencial primaria -> sesión pre-MFA + desafío(user, session, TTL)
                    -> verificación OTP -> consumir desafío -> rotar/elevar sesión
                    -> autorización del recurso
```

Superficie de ataque

Login, recuperación, cambio de factor, "remember device", reautenticación de acciones sensibles, resend y endpoints API equivalentes al paso visual.

Cómo identificarla

- Acceso directo al recurso con sesión pre-MFA.
- Bandera cliente cambia la transición sin OTP válido.
- Código de A funciona en sesión B.
- OTP aceptado más de una vez o después de resend/expiración.
- Intentos ilimitados o contadores no ligados a cuenta/desafío.

Preguntas que debo hacerme

1. ¿Qué estado tiene la sesión antes y después?
2. ¿A qué usuario, sesión y desafío está ligado el OTP?
3. ¿Se consume al usarlo y al reenviar?
4. ¿Qué recurso prueba realmente la transición?
5. ¿Hay fallback o recuperación más débil?

Prueba mínima

Con cuentas de laboratorio, comparar OTP inválido, válido, reutilizado, expirado y de otra sesión. Verificar el recurso protegido, no solo la redirección.

Construcción progresiva del payload

La secuencia mínima conserva cookie pre-MFA, envía código inválido con y sin campos adicionales y consulta el recurso protegido. Después usa un código válido una vez y repite exactamente la solicitud para probar replay.

Anatomía de los payloads

- Contexto de entrada: JSON /mfa/verify con cookie pre-MFA.
- Sintaxis original: {"otp":"000000","verified":false}.

- Entrada controlada: OTP y bandera cliente.
- Transformaciones conocidas: JSON decode y lookup de desafío.
- Parser final: máquina de estados de autenticación.
- Sink: elevación de sesión.
- Primitiva: elevar con OTP inválido cambiando solo `verified`.
- Payload mínimo: `{"otp": "000000", "verified": true}`.
- Significado de cada componente: código deliberadamente inválido; bandera reclama un estado.
- Resultado esperado: rechazo y sesión pre-MFA sin cambios en una implementación segura.
- Control negativo: mismo OTP con `false` y acceso directo al recurso.
- Restricción observada: endpoint ignora el código cuando `verified=true`.
- Por qué falla la variante básica: enviar solo OTP inválido no alcanza la rama defectuosa.
- Hipótesis de adaptación: mass assignment/estado cliente confiado.
- Payload adaptado: añadir solo la propiedad observada en el esquema.
- Por qué debería funcionar: el backend vulnerable la enlaza al objeto de estado.
- Evidencia: nueva sesión accede al recurso pese a OTP inválido.
- Cuándo no funcionaría: DTO cerrado y estado derivado exclusivamente del verificador.

Variaciones según el contexto

TOTP, código por correo, push y WebAuthn tienen propiedades distintas. El análisis común es binding, frescura, consumo, intentos y transición. “Recordar dispositivo” crea otro autenticador persistente que debe evaluarse por separado.

Filtros y bypasses

Cambiar nombres de bandera al azar no es metodología. Deriva campos del tráfico o esquema, modifica uno y comprueba estado server-side. Rate limiting, replay y bypass de transición son hallazgos distintos.

Evidencias de confirmación

Acceso al recurso o sesión elevada sin factor válido; aceptación cruzada entre usuarios/sesiones; reutilización del mismo OTP cuando debería ser de un solo uso. Ver la pantalla de OTP o saltarla visualmente no basta.

Escalado de impacto

Probar solo cuentas y desafíos propios, medir límites sin agotamiento abusivo y evaluar recuperación/cambio de factor con datos de laboratorio.

Errores frecuentes

- Concluir bypass por recibir 302.
- No conservar la cookie pre-MFA correcta.
- Confundir ausencia de rate limit con bypass inmediato.
- Reutilizar un OTP dentro de una ventana sin conocer política de consumo.
- Probar código de otro usuario sin controlar el binding de sesión.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
verified=true ignorado	DTO cerrado o nombre incorrecto	esquema/traza y propiedad desconocida como control
dashboard carga, API privada da 401	solo salto visual	consultar recurso protegido crudo
OTP válido de A falla en B	binding correcto o TTL	misma sesión A como control
OTP se acepta dos veces	replay o dos desafíos	comparar challenge ID y evento de consumo
resend deja ambos códigos válidos	invalidación incompleta	usar viejo/nuevo en orden controlado
intentos se reinician con cookie	contador ligado a sesión	misma cuenta con sesión nueva, sin exceder límites

Mitigaciones

Estado server-side, DTO cerrado, binding a usuario/sesión/desafío, TTL corto, uso único, límites por cuenta y desafío, rotación de sesión tras MFA, recuperación equivalente y registros sin OTP.

Relación con pentesting y certificaciones

La competencia es demostrar una transición inválida y su efecto de autorización, no adivinar códigos.

Caso guiado

Caso A — Básico: estado cliente confiado

Con OTP inválido, verified=false rechaza y verified=true entrega una cookie nueva que accede a /account.

Observación: una propiedad cambia estado. Qué sé: el código sigue inválido. Hipótesis: backend confía en bandera o existe caché. Experimento: cookies separadas, token de desafío único y acceso directo. Resultado: solo la bandera eleva la sesión. Conclusión: bypass de MFA por manipulación de estado. Siguiendo paso: documentar binding y rotación.

Caso de adaptación

Caso B — La bandera no funciona, pero el código se reutiliza

El DTO rechaza propiedades extra. Un OTP válido se acepta dos veces en dos sesiones del mismo usuario.

Observación: mass assignment descartado; posible replay. Hipótesis: mismo desafío compartido o consumo ausente. Experimento: registrar challenge ID, usar código una vez y repetir antes de expirar. Resultado: mismo challenge aceptado dos veces. Conclusión: fallo de uso único, no manipulación de parámetro. Siguiendo paso: probar invalidación tras resend.

Caso C — Transferencia: cambio de teléfono

El cambio de factor crea un desafío, pero una sesión pre-MFA de login puede enviarlo y modificar el número. El título no revela la técnica.

Resolución: sesión pre-MFA -> endpoint de gestión -> autorización insuficiente. La primitiva transferida es una transición sensible accesible en estado incorrecto, no un OTP adivinado.

Caso D — Falso positivo: ruta /dashboard

Tras OTP inválido, el servidor redirige a /dashboard, cuyo HTML carga, pero todas las API privadas responden 401 y la página muestra "sesión incompleta".

Experimento: recurso protegido y estado /me. Resultado: sesión sigue pre-MFA. Conclusión: navegación no equivale a bypass.

Ejercicios

1. Diseña una matriz OTP inválido/válido/replay/expirado y sesión A/B.
2. Distingue rate limiting débil de bypass de transición.
3. Explica cómo probar binding sin usar cuentas ajenas.
4. Modela resend y consumo del desafío.

Resumen

MFA es una máquina de estados ligada a un desafío. La evidencia es el estado autorizado obtenido, no la pantalla ni la redirección.

Chuleta operativa

1. Sesión pre-MFA.
2. Challenge ID, usuario y TTL.
3. OTP inválido/válido/replay.
4. Propiedades extra derivadas del esquema.
5. Recurso protegido.
6. Rotación, consumo y límites.

Referencias

- [OWASP MFA Cheat Sheet](#)
- [NIST SP 800-63B](#)
- [Concepto interno](#) (mfa-otp-bypass)

Navegación

Anterior: [File upload](#). Siguiente: [Autenticación y sesiones](#). Índice: [curso](#).